设计文档

指令解析

代码要求实现两个完整 128 位向量寄存器中 16 对 signed int8 的点积。若指令记作

vdot.vv vd, vs2, vs1

以下编码与语义用于本作品的 RTL、软件参考和测试：

```
S = sum(signed8(vs1[8*i+7:8*i]) * signed8(vs2[8*i+7:8*i]), i=0..15)
vd[31:0] = two_complement_32(S)
vd[127:32] = 0
```

指令覆盖目的寄存器，不累加旧 vd。且仅在低 32 位保存补码，高 96 位保持为零；例如 S=-1 时结果为 0x00000000000000000000000ffffffff。vxrm 和 vxsat 不参与计算且保持原值。

约束	代码
VLEN / SEW / LMUL	128 位 / 8 位 / m1（编码 000）
vl / vstart / vm / vill	16 / 0 / 1 / 0
尾部与掩码策略	合法配置下结果完整覆盖 128 位
向量状态 VS=Off	复用香山原有向量状态检查，报告 illegal instruction

以下为指令编码

位	31:26	25	24:20	19:15	14:12	11:7	6:0
内容	000011	vm	vs2	vs1	000	vd	1010111

编码占用 OPIVV 下的实验 funct6=000011。vm 在译码中通配，使 vm=0 进入明确的非法配置处理；合法发射只接受 vm=1。

```
decode: (inst & 0xfc00707f) == 0x0c000057
vm=1:   (inst & 0xfe00707f) == 0x0e000057
vdot.vv v10,v8,v9 -> 0x0e848557
FuType = vipu, fuOpType = 9'h0fe
```

数据通路选择合适的逻辑。signed int8 的取值为 -128～127。单项乘积最大为 (-128)×(-128)=16384，最小为 (-128)×127=-16256，16 位带符号数可容纳单项乘积。16 项最大和为 +262144，最小和为 -260096。19 位带符号正数上限只有 +262143，最终内部累加必须使用 20 位。

运算层	数量	带符号位宽	寄存位置
逐元素乘法	16	8×8 → 16	组合逻辑
相邻两项相加	8	16 +& 16 → 17	第一级：pairSumsReg
四项部分和	4	17 +& 17 → 18	组合逻辑
八项部分和	2	18 +& 18 → 19	组合逻辑
完整点积	1	19 +& 19 → 20	组合逻辑

运算层	数量	带符号位宽	寄存位置
结果格式化	1	20 符号扩展为 32，再拼接 96 位零	第二级：result

先用位切片得到 8 位元素，再调用 asSInt。若先把整个 128 位向量解释成带符号数再移位，元素符号容易被错误传播。乘法两端均为 SInt，产生 16 位结果。加法使用 +& 逐层保留进位；普通 + 不会自动为每层增宽。

balancedSum 在 chisel 生成期递归，展开后为固定平衡加法树。16 个乘法器与 15 个加法器完成一条指令的算术；前 8 个加法器放在第一级寄存器之前，后 7 个在第二级之前。采用平衡加法树减少串行累加层数。

signedResult 声明为 SInt(32.W)，赋值时对 20 位 sum 做符号扩展。signedResult.asUInt 改变解释方式，再用 Cat 拼接 96 位零。

香山适配与 VIPU 流水

VecDecoder 的 OPIVV 表增加一项，将实验编码路由至 FuType.vipu，启用向量目的寄存器写入。该路径复用向量寄存器读写、重命名、调度、ROB 和异常提交机制。合法 m1、vl=16 配置使用完整 128 位源寄存器。

KksVDot.scala 集中保存 BitPat、编码常量、9 位 fuOpType 和 unsupported 配置函数。

所在级	判定与动作	与指令对齐的原因
输入至第一级	regEnable(1) && isDot16	按当前输入的 fuOpType 接收乘积部分和
第一级至第二级	regEnable(2) && ctrlVec(1).fuOpType==opType	读取上一拍控制，防止相邻标准 VIPU 运算误推动 dot 结果
输出选择	outCtrl.fuOpType==opType	以输出级指令选择 dot16 或原 VIAlu 数据
输出异常	outVecCtrl / outVl / outVstart / outVm	配置和输出数据属于同一条指令

regEnable 来自 HasPipelineReg 中的有效位与就绪信号。VDot16 本身只保存数据，不重复创建 valid、目的寄存器编号或 ROB 编号。输出有效、控制传递和冲刷继续使用既有 VIPU 接口。算术寄存器没有复位初值，输出仅在相应控制有效时被消费。

dot 指令进入时关闭原 vialu 的 in.valid；标准运算继续使用原路径。输出通过 outCtrl 进行选择，避免用当前输入指令判断两拍前的结果。

unsupported 同时检查 vill、vsew、vlmul、vl、vstart 和 vm。检查点在输出级，异常进入既有 illegalInstr 通路。VS=Off 由香山原有前端检查负责。本模块可在内部计算非法指令的数据，但异常指令不应提交；整核非法配置测试检查异常原因、异常 PC 和旧目的寄存器是否保持。

源码位于 rtl/VDot16.scala、rtl/KksVDot.scala 和 patches/xiangshan.patch；新增模块由脚本复制到对应 package。Funct6、fuOpType 或支持范围变化时。

接口方面，VDot16 的输入为 vs1、vs2 两个 UInt(128.W)，控制为 enableS1、enableS2，输出为 UInt(128.W)。独立模块不访问 CSR，不使用旧目的寄存器，不发起内存操作。接口可迁入其他 128 位向量流水，且当前实现只支持固定的 VLEN、SEW、LMUL 和 vl。VDot16Layout 集中给出位宽。

构建与复现操作

操作均在正式作品根目录执行。依赖来自同级 xs-env；异地复现可先设置 XS_ENV 指向已部署的 xs-env。env/versions.json 保存基础源码及子模块；prepare_sources.py 在 work 中建立隔离工作树。所有新增实现和测试都包含在本作品中。

```
cd /home/hbyasts/2-corecode/riscv-xiangshan/cie2026-xvkongkabishourushishuo-vdot-for-riscv
source scripts/env.sh
bash scripts/test_chisel.sh
bash scripts/build_emu.sh
bash scripts/build_app.sh vdot
bash scripts/build_app.sh illegal
bash scripts/build_app.sh vadd
KKS_TIMEOUT=3600 bash scripts/run_emu.sh vdot custom nodiff
KKS_TIMEOUT=1800 bash scripts/run_emu.sh illegal custom nodiff
bash scripts/run_emu.sh vadd custom nodiff
```

cpu 配置为 KunminghuV2Config，构建参数保留 EMU_TRACE=1、WITH_CHISELDB=1，该仿真器支持跟踪。环境使用 Ubuntu 22.04、JDK 21、Mill 0.12.3、Chisel 6.6.0、firtool 1.62.1、Verilator 5.052 和 RISC-V GCC 16.1。单元测试使用 Treadle；cpu 仿真使用 Verilator。当前使用 custom nodiff：执行新构建的 RTL，测试程序用独立的 C 标量参考比较指令结果。未来再为 NEMU 添加同一指令并启用 DiffTest。

单元测试检查编码、配置、所有 signed-byte 乘数对、每条 lane 以及独立使能暂停。整核测试覆盖正数、负数、边界值、固定种子随机输入、寄存器重叠、CSR 保持、连续依赖、标准 VIPU 指令交错、分支跳过非法编码和非法配置。保留的完整日志位于 stages/02-vdot/logs。整核构建成功后，build/emu-build-manifest.json 记录 Chisel、补丁、生成 RTL 。包括：单元测试.log,基础功能测试.log,非法配置测试.log,向量指令测试.log。